

HyCaMi: High-Level Synthesis for Cache Side-Channel Mitigation

Heiko Mantel[✉], Joachim Schmidt[✉], Thomas Schneider[✉], Maximilian Stillger[✉], Tim Weißmantel[✉],
Hossein Yalame[✉]

Technical University of Darmstadt, Germany

{mantel, weissmantel}@mais.informatik.tu-darmstadt.de

{joachim.schmidt, maximilian.stillger}@stud.tu-darmstadt.de

{schneider, yalame}@encrypto.cs.tu-darmstadt.de

ABSTRACT

Cache side-channels are a major threat to cryptographic implementations, particularly block ciphers. Traditional manual hardening methods transform block ciphers into Boolean circuits, a practice refined since the late 90s. The only existing automatic approach based on Boolean circuits achieves security but suffers from performance issues. This paper examines the use of Lookup Tables (LUTs) for automatic hardening of block ciphers against cache side-channel attacks. We present a novel method combining LUT-based synthesis with quantitative static analysis in our HyCaMi framework. Applied to seven block cipher implementations, HyCaMi shows significant improvement in efficiency, being 9.5× more efficient than previous methods, while effectively protecting against cache side-channel attacks. Additionally, *for the first time*, we explore balancing speed with security by adjusting LUT sizes, providing faster performance with slightly reduced leakage guarantees, suitable for scenarios where absolute security and speed must be balanced.

ACM Reference Format:

Heiko Mantel[✉], Joachim Schmidt[✉], Thomas Schneider[✉], Maximilian Stillger[✉], Tim Weißmantel[✉], Hossein Yalame[✉]. 2024. HyCaMi: High-Level Synthesis for Cache Side-Channel Mitigation. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3657394>

1 INTRODUCTION

Cache side-channels are unintended flows of information across system layers. In a cache side-channel attack, an adversary obtains information on secret inputs by observing the minute timing differences caused by cache hits and misses triggered by runs of the target program. Despite well-known countermeasures, prevalent cryptographic libraries like *OpenSSL*, *mbedtls*, and *Nettle* often rely on implementations of AES-256 that are susceptible to such attacks, especially in the absence of specialized hardware like AES-NI.

In response to these vulnerabilities, developers have historically employed constant-time programming methods to manually harden cryptographic implementations. This approach is evident in the creation of bitsliced versions of DES [5] and AES-GCM [17], which are acknowledged for both their resistance to side-channel attacks and

high performance. These secure versions are crafted using hand-designed Boolean circuits that inherently possess constant-time properties. However, this manual process is not foolproof and is susceptible to introducing new vulnerabilities, either through human error or unforeseen compiler optimizations. For instance, [9] has identified leaks in constant-time code running in production.

The pursuit of automatic side-channel hardening tools has become a focal point in recent research. Projects like Raccoon [25], SC-Eliminator [31], and Constantine [6] represent significant strides in this direction, as outlined in our literature review (see Table 1). However, while these tools make compelling cases for their effectiveness in enhancing side-channel security, they often lack comprehensive static verification of the security properties in the resulting binaries. In contrast *RiCaSi* [21] is more closely aligned with our work, as it utilizes a methodology similar to the initial manual hardening approach. It automatically generates Boolean circuits from C implementations, akin to the early manual methods, then translates them back to C and compiles to x86. This process allows for static analysis verification, ensuring that the resulting binaries are free from cache side-channels. However, a critical limitation of *RiCaSi* is the significant performance degradation it introduces, as the binaries generated are considerably slower compared to their original versions, highlighting a trade-off between security and efficiency.

Our research addresses this performance-security trade-off by adopting a shift similar to that seen in Multi-Party Computation (MPC): from Boolean circuits to Lookup Tables (LUTs). LUTs have been recognized as essential components in various computational areas, including cryptography and secure computation, especially in MPC, where LUT-based techniques have overwhelmingly been preferred over traditional Boolean circuit evaluations [7, 10]. Our work aims to provide an automatic side-channel hardening solution that not only retains relative speed but also offers robust, statically verifiable security guarantees through the use of LUTs.

1.1 Related Work

This section provides a concise overview of related works.

Vulnerabilities in Block Cipher Implementations: Despite being a known risk for cache side-channel vulnerabilities, popular block cipher implementations like AES-256 from *OpenSSL* [13] and DES, 3DES, and Camellia from *mbedtls* [19] continue to use large lookup tables for computational efficiency. These are the default choices in the absence of hardware acceleration. Examples for types of cache side-channel attacks include measuring runtime [3], learning which cache lines have been evicted by the target program [14], abusing access latency to shared memory buffers [32] and analyzing cache hits and misses from power traces [4]. Traditional mitigation involves constant-time programming techniques, such

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

<https://doi.org/10.1145/3649329.3657394>

Tool / Implementation	Obfuscation Technique	Automatic Hardening	Verified Leakage	No Hardware Extension	Partial Hardening
Bitsliced DES (FSE'97)	bitslicing	✗	✗	✓	✗
Bitsliced AES-GCM (CHES'09)	bitslicing	✗	✗	✗ (Intel SSE)	✗
Raccoon (USENIX'15)	transactional + ORAM	✓	✗*	✗ (AVX)	✗
SC-Eliminator (ISSTA'18)	transformation + pre-loading	✓	✗*	✓	✗
RiCaSi (CANS'20)	circuit compilation	✓	✓	✓	✗
Constantine (CCS'21)	linerization	✓	✗*	✗ (AVX)	✗
HyCaMi	Secure LUTs	✓	✓	✓	✓

Table 1: Comparison to other Hardening Approaches (*performance optimizations prevent verification of hardened program)

as bitslicing [5] which involves manually translating a block cipher specification to a circuit representation of logical bit operations. This manual approach has later been optimized for throughput using SIMD instructions for parallelism [17].

Static Cache Side-Channel Assessment: Qualitative tools like *CacheS* [28] and *CaType* [16] focus on the detection and localization of side-channel vulnerabilities. In this work we build on the family of tools *CacheAudit*, a quantitative approach that provides information theoretic upper bounds on the cache side-channel leakage of x86 binaries. The approach and initial implementation was published in [12] and later extended by the original authors [11] and authors of this paper [21, 22, 29]. Other quantitative tools include works on timing-attackers without cache on the level of C [15] and Java [20].

LUT-Based Circuits in Secure Multiparty Computation: In Secure Multiparty Computation (MPC), LUT-based protocols like FLUTE [7] and LUC [10] have enhanced efficiency by reducing cryptographic operations and circuit complexity. The tools for creating LUT-based circuits primarily originate from hardware synthesis targeting FPGAs. Yosys [30], an open-source framework, maps Verilog designs into circuits for various hardware targets, and has been employed in an MPC context [24]. To bridge the gap for programmers accustomed to imperative languages, high-level synthesis tools like XLS [1] from Google have been developed, which compile C/C++ code into Verilog, thus facilitating the design of MPC circuits in familiar programming languages.

This work: Following the exploration of LUTs in MPC as previously discussed, we now focus on the following question:

Are programs automatically hardened using LUTs more efficient in performance compared to those automatically hardened with Boolean circuits, while retaining the same cache side-channel security?

To address this question, we introduce HyCaMi, an innovative approach that merges LUT synthesis with quantitative side-channel analysis. We proceed by elaborating on our exact contributions.

1.2 Our Contributions

In this work, we pivot the traditional understanding of LUTs in the realm of cybersecurity, transitioning from their conventional role as a vulnerability in side-channel attacks against block ciphers to a robust defensive mechanism. This novel application necessitates the development of new, optimized LUT-based circuit representations. Acknowledging the complexity and error-proneness of manual construction, we introduce an innovative automated toolchain. This toolchain adeptly transforms high-level function

descriptions into efficient multi-input, multi-output LUT representations, leveraging repurposed hardware synthesis tools beyond their original purposes. Our approach, while producing binaries that may be slower compared to existing methods like Raccoon [25], SC-Eliminator [31], and Constantine [6], achieves superior security guarantees via static analysis of the hardened binaries. Compared to *RiCaSi* [21], the only other tool with comparable security properties on the binary, we achieve a speed that is up to 9.6× faster. Our main technical contributions are summarized as follows:

- **Encoding LUT-Based Circuits into C:** We introduce a cutting-edge method for encoding LUT-based circuits into C. This method strategically balances security and performance, offering either complete security against cache side-channels or significantly minimizing cache side-channel leakage, depending on the LUTs' size.
- **Development of HyCaMi Framework:** We develop HyCaMi, a comprehensive framework for the automatic hardening of block ciphers. HyCaMi synergizes LUT-based high-level synthesis with quantitative static side-channel analysis, culminating in a pipeline that automates the hardening of C/C++ source code. Our framework is open-sourced at <https://encrypto.de/code/HyCaMi>.
- **Extensive Evaluation Across Multiple Implementations** Our framework's efficacy is rigorously tested across four AES-256 implementations from *OpenSSL*, *mbedtls*, *Nettle*, and *LibTomCrypt*, and implementations of DES, 3DES, and Camellia from *mbedtls*. We demonstrate that HyCaMi produces binaries that are free of cache side channels and which are up to 9.6× faster than the hardened binaries of state-of-the-art work [21].
- **Exploring Security-Runtime Trade-Offs:** We explore, for the first time, the security-runtime trade-off induced by increasing size of LUTs. Applied to the same block ciphers, we show that in this configuration the binaries are up to 4.5× faster when compared to the fully secure variants, while for access-based attackers only having at most 18% of the leakage bound of the original program.

2 PRELIMINARIES

In this section we give a quick overview on the automatic quantification of cache side channels using program analysis. We calculate an upper bound of the capacity of a discrete memory-less channel $C: I \rightarrow O$, where I is a finite set of secret inputs, O is a finite set of side-channel outputs and C models the behavior of the target program. Leakage is defined as the difficulty of guessing the secret input given the side-channel output. In other words, leakage is the difference in uncertainty of the attacker over the secret input before and after seeing the side-channel output. If this difference in

uncertainty is given in terms of min-entropy (called min-entropy leakage), this value provides a measure on the reduction of guesses for a one-shot attacker in bits [26]. As also noted in [26], an upper bound of the min-entropy leakage can be calculated by counting the number of possible side-channel observations (i.e. elements of O). The family of tools *CacheAudit*, applies this approach to cache side channels on the x86 architecture. The tool overapproximates the set O by applying abstract interpretation [8] with an abstract model of the x86 architecture with cache. We categorize four distinct models of cache-side-channel adversaries, identified as *acc*, *accd*, *trace*, and *time*. They are:

- O^{acc} : The set of all cache states after termination of the victim program. In this model the attacker can deduce which memory blocks of the victim program are cached in a shared cache.
- O^{accd} : The set of all cache states after termination of the victim program. This attacker is similar to *acc*, but can only deduce how many memory blocks the victim program has loaded in each cache set of a shared cache.
- $O^{trace} \subseteq \{hit, miss, none\}^*$: The set of sequences (traces) of cache interactions. These include all instances of cache hits, misses, and cases of 'no access'. Such traces offer a detailed view of the cache behavior throughout a program's execution.
- $O^{time} \subseteq \mathbb{N}$: The set of possible running times as influenced by the caching behavior. These times are calculated for a fixed duration for cache hits, misses, and non-memory accessing instructions.

These adversary models have been previously implemented for the *CacheAudit* family of tools. This pre-existing implementation allows us to integrate these models into our framework seamlessly, without necessitating any modifications to the existing code.

3 SECURE LUTS IN C

Whether a LUT with secret-dependend accesses is a potential cache-side-channel vulnerability or not depends on the size of the LUT, how it is positioned in memory and how it is accessed by the program. In this section we present a novel technique for placing and accessing LUTs that is either fully side-channel secure, or minimizes leakage. The technique works for multi-input multi-output LUTs with at most 8 inputs and outputs. For this section we assume a target cache with 32 KiB size and a line size of 64 byte, the specification for a typical L1 data cache of Intel 8th Gen or AMD Zen 3 desktop chips. Only minor changes are required to adapt this technique to other caches of different size.

We distinguish two cases. If all LUTs have at most 6 inputs, we align them to 64 byte boundaries (i.e. the size of one cache line). If any LUT can have more than 6 inputs, we take two precautions to minimize leakage. First we place all LUTs at 32KiB boundaries (i.e. the size of the cache). Second, we rearrange the values in each table such that every access refers to the same cache set. For the cache described above, we place every value within 64 byte blocks that are located 4KiB apart (i.e. the number of cache sets multiplied by the cache line size). Listing 1 shows the source code used for an example 8-input 6-output LUT that was hardened.

Following this method, if all LUTs have 6 inputs or less, the hardened binary is fully secure against cache side-channels by design. This is because every table takes $2^6 \cdot 8 \text{ bits} = 64 \text{ bytes}$ in memory and thus fits into one cache line. Hence, even if the access

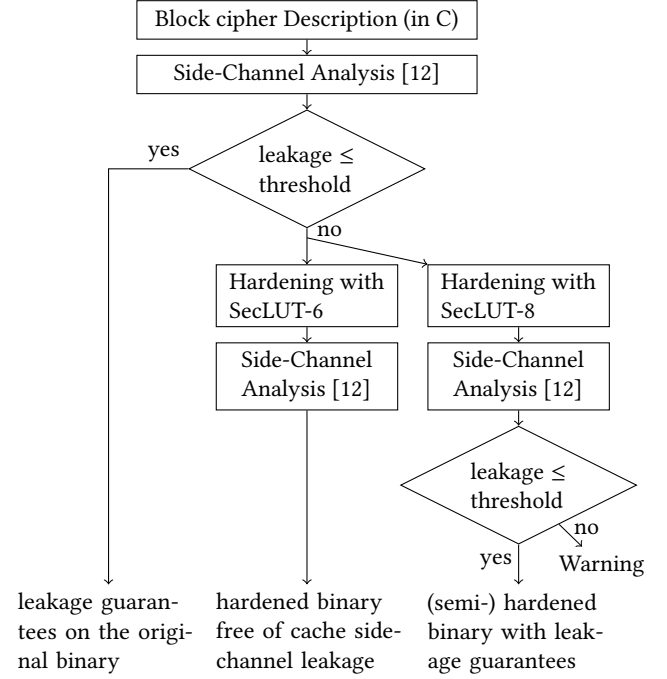


Figure 1: Workflow of HyCaMi

location to the LUT is secret-dependent, no leakage occurs. LUTs with more than 6 inputs do not fit into one cache line. To minimize leakage we place them such that every access goes to the same cache set. For example, without hardening, a LUT with 8 inputs takes $2^8 \cdot 8 \text{ bits} = 256 \text{ bytes}$ of memory and thus span four cache lines. After the hardening step, each 64 byte block is placed 4KiB apart such that all four cache lines fall into the same cache set. The result is that all LUT accesses are cached in the same cache set. Thus, if enough different LUTs are accessed (e.g. 8 for the LRU replacement strategy) the cache lines of prior LUT accesses are evicted, thereby hiding information from a cache side-channel attacker. For an access-based attacker without shared memory (i.e. attackers modeled by *accd*), this limits the amount of possible different observable states to 9, or at most $\log_2(9) \approx 3.17$ bits of leakage. We can not make such predictions for the other three attacker models.

Listing 1: Excerpt of generated C-code accessing tables for SecLUT-8.

```

static const uint8_t table_3[] __attribute__((aligned
(32768))) = {0b00111111, ...};
...
uint32_t addr_3 = wire_0 | wire_2<<1 | wire_1<<2 | wire_3
<<3 | wire_5<<4 | wire_7<<5 | wire_6<<6 | wire_4<<7;
addr_3 = ((addr_3 >> 6) * 4096) + (addr_3 % 64);
uint8_t tmp_3 = table_3[addr_3];
uint32_t wire_24 = tmp_3 & 0b1;
uint32_t wire_25 = (tmp_3 & 0b10) > 0;
uint32_t wire_26 = (tmp_3 & 0b100) > 0;
uint32_t wire_27 = (tmp_3 & 0b1000) > 0;
uint32_t wire_28 = (tmp_3 & 0b10000) > 0;
uint32_t wire_29 = (tmp_3 & 0b100000) > 0;

```

4 THE HyCaMi FRAMEWORK

The general workflow of HyCaMi is depicted in Figure 1. Given a C implementation of a block cipher that is possibly vulnerable

to cache side-channel attacks, the source code is compiled to x86 machine code and statically analyzed with *CacheAudit* v0.3 [12, 21]. If the leakage bound is acceptable, no hardening is required and the framework simply supplies these leakage guarantees on the binary. If, however, the leakage bound is too high, there are two paths. In both paths, the C source code is translated into an equivalent function following our synthesis flow. For the default case, LUTs with at most 6 inputs are chosen, which results in a fully secure program (cf. Section 3). We call this configuration SecLUT-6. Alternatively, the synthesis flow is applied using LUTs with at most 8 inputs. This results in a program that is potentially leaky with likely faster run times. We call this configuration SecLUT-8.

Both binaries are analyzed with *CacheAudit*. For the fully hardened version, this step statically verifies that the compiler did not introduce new unexpected leakage. In the default case, this verification passes. For the SecLUT-8 version, this step attests whether the binary may be suited for the chosen scenario. If not, the framework still supplies this leaky variant together with a warning.

Synthesis: On a high level, *HyCaMi* translates a C implementation of a function into an equivalent C implementation based on secure LUT access by using open-source frameworks integrated into a custom compilation workflow. An overview of all processing steps is given in Figure 2. First, the function is manually extracted from the source C file. Depending on the implementation of the function, it may be necessary to modify the source code manually and replace C/C++ constructs not supported by XLS [1]. The functions studied in this paper required the conversion of unbounded while loops containing break statements to bounded for loops. Additionally, the implementations use pointer arithmetic which were replaced with array indexing operators. Given an implementation in C/C++ containing only supported constructs, XLS [1] translates the C/C++ code into an intermediate representation (IR) of a Boolean circuit along with additional metadata. This metadata describes the translation of the function signature in C to the circuit inputs and outputs in the IR. After applying optimizations, XLS translates the IR into Verilog code. The Verilog code generator is configured to output combinational Verilog circuits instead of pipelined circuits. Pipelined circuits achieve higher throughput by exploiting parallelism in hardware designs. The evaluation of the resulting LUT circuit happens in a single thread in software, thus the benefits of pipelining do not apply.

We use Yosys [30] to synthesize the Verilog code into Boolean circuits containing multi-input, single-output LUTs. This circuit containing multi-input, multi-output LUTs is translated into C code using our novel LUT2C converter. The LUT2C translator requires information on the cache architecture of the target processor in order to layout the lookup table bitstring in a way which minimizes the information revealed in side-channels about the table index accessed. The resulting C source code is divided into three parts: The unwrapping of function parameters into individual wire values, the calculation of the results, and the wrapping of wire values back into C types. The unwrapping and wrapping code is generated using the metadata provided by XLS in the first step of the compilation pipeline. This makes it possible for the generated function to have the same function signature as the original implementation.

The calculation of the result consists of a series of operations for each LUT gate. First, the index into the lookup table data is

calculated by concatenating the input wire value bits. The data in the corresponding index is then read from a byte array. Depending on the number of outputs, the bits contained in the byte are assigned to multiple wires. We describe the access in detail in Section 3. The generated code can then be compiled into a binary object file and linked with the original code with a C/C++ compiler.

5 PERFORMANCE EVALUATION

We evaluate the effectiveness of hardening and the overhead incurred by *HyCaMi* at the example of seven block cipher software implementations. First, we apply *HyCaMi* to DES, 3DES and Camellia from *mbedtls* 2.16.5 [19]. All three implementations use LUTs to speed up computation. Such LUT-based implementations of DES, and Camellia are known to be vulnerable to cache side-channel attacks [27]. Second, we apply *HyCaMi* to AES-256 implementations from *OpenSSL* 1.1.1d [13], *mbedtls* 2.16.5 [19], *Nettle* 3.5 [23] and *LibTomCrypt* 1.18.2 [18]. Despite implementing the same block cipher, the library authors utilized LUTs of different sizes, resulting in different cache-side-channel leakage characteristics [22]. For all block ciphers, we consider the implementation of the key schedule and the encryption algorithm. For the hardening, we have extracted all LUTs into individual functions and replaced all LUT accesses with calls to these functions. We then apply our new method to these extracted functions as described in Section 4.

Setup: For the evaluation we use an Intel i9-10900K, considering an attacker with access to a process running on the same system observing a shared L1 data cache. This cache has a size of 32 KiB, a line size of 64 byte and an associativity of 8.¹ We assume an LRU cache line replacement policy, since to the best of our knowledge, the replacement policy is not publicized for this CPU. We confirm the functional correctness of the hardened AES-256 binaries using test vectors from the NIST Cryptographic Algorithm Validation Program [2]. For the DES, 3DES and Camellia binaries we compare the output of the original program to the hardened variants for 10.000 random generated inputs.

5.1 Evaluation of generated Circuits / LUTs

In Table 2, we give a brief overview of the distribution of different types of LUT in the generated circuits. A difference between DES/3DES and the other two block ciphers is immediately visible. AES and Camellia both use 8-input, 8-output S-boxes, whereas DES uses 6-input, 4-output S-boxes. In the AES implementations we analyzed, these S-boxes are not used directly, but combined with other operations into larger 8-input 32-output LUTs for increased performance. We thus conclude that these underlying differences in the structure lead to different characteristics in the LUT distribution.

We can also see how the number of possible inputs affects the distribution and overall number of gates in the resulting circuit. Due to allowing for more inputs to be gathered in one gate, the 8-input LUT circuits have consistently lower gate counts. Averaged over the four AES implementations tested, the 6-input circuits have ~ 14× more LUTs. The 6-input circuits for Camellia and DES/3DES are 8.4× and ~ 4.3× larger, respectively. This has an effect on the runtime performance, which is discussed in Section 5.2. Also noteworthy are the high numbers of 1-input, 6-output and 1-input,

¹This information was obtained using the command-line tool *lstopo*.

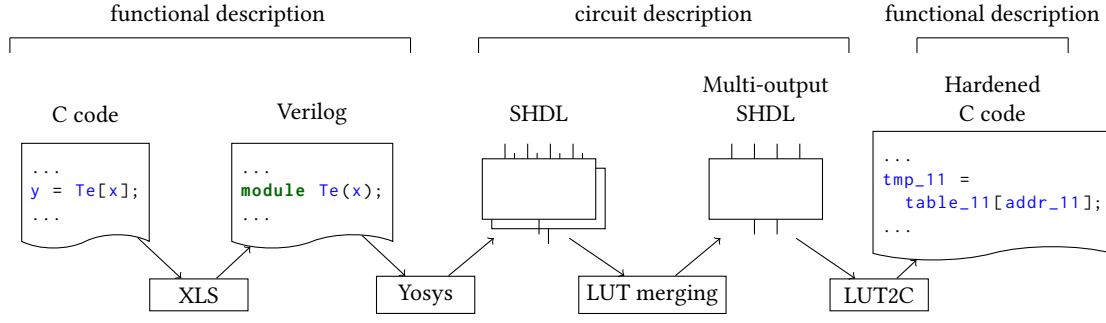


Figure 2: Synthesis flow

	Library	Cipher	Inputs/Outputs				
			4/1	6/1	6/2	6/5	other
SecLUT-6	mbedtls	DES	0	150	21	3	85
	mbedtls	3DES	0	150	21	3	85
	mbedtls	Camellia	60	0	16	20	72
	mbedtls	AES-256	167	0	26	28	121
	OpenSSL	AES-256	152	0	22	25	100
	Nettle	AES-256	167	0	26	28	121
	LibTomCrypt	AES-256	170	0	24	29	120
SecLUT-8	mbedtls	DES	20	10	10	0	20
	mbedtls	3DES	20	10	10	0	20
	mbedtls	Camellia	0	0	8	8	4
	mbedtls	AES-256	0	4	2	6	13
	OpenSSL	AES-256	0	4	0	4	12
	Nettle	AES-256	0	4	2	6	13
	LibTomCrypt	AES-256	0	4	0	4	17

Table 2: Distribution of LUT gate sizes in generated circuits

7-output gates. As a 1-input LUT can only have 2 distinct outputs, further output wires are redundant. This issue occurs because the LUT merger treats individual LUTs as black boxes, making it unable to group output wires together based on their function. Removing this limitation is worth investigating in the future.

5.2 Runtime Overhead

Table 3 illustrates the runtime analysis of programs hardened with 6-input and 8-input LUTs, compared to their original versions. This analysis involved averaging the runtime over 1,000,000 executions of the key schedule and encryption functions for each covered implementation, further averaged across 20 repetitions to ensure consistency. The results confirm a significant increase in runtime overhead due to the hardening process, particularly impacting ciphers like AES-256 with larger original LUTs more than those with smaller LUTs, such as DES and 3DES. Notably, the data also reveals that the binaries hardened with the SecLUT-8 configuration exhibit enhanced performance, being 2.6× to 4.5× faster than those in the SecLUT-6 configuration, thereby indicating that larger LUTs in the hardening step contribute to improved runtime efficiency.

5.3 Leakage Assessment

To evaluate the cache side-channel leakage we use the static analysis tool *CacheAudit* v0.3. For both case studies we analyze wrapper programs that first set up library specific data structures and then call the key schedule and encryption function. The key and message are left uninitialized such that *CacheAudit* considers them as private input with unknown value. The wrapper programs are compiled with *gcc* 9.3.0 using the parameters *-m32-fno-stack-protector* to obtain x86 binaries compatible with the tool. The analysis results are depicted in Table 4. As expected the results for the original programs reveal potential side-channel leakage across all considered block cipher implementations. The evaluation confirms for the examples the SecLUT-6 configuration is fully secure against cache side-channels, while programs hardened via the SecLUT-8 configuration are still potentially leaky but have reduced bounds when compared to the original program. Moreover, as predicted the leakage bounds for *accd* stayed below 3.17 bits. We therefore have shown that HyCaMi offers two types of hardening depending on developer needs. Either the framework generates a fully secure binary or a faster binary is leaky but still has better side-channel security properties than the original program.

Library	Cipher	Orig.	SecLUT-6	SecLUT-8
mbedtls	DES	0.25 μ s	6.0 μ s/ 24.0×	2.3 μ s/ 9.2×
mbedtls	3DES	0.61 μ s	17.8 μ s/ 29.5×	6.9 μ s/11.3×
mbedtls	Camellia	0.32 μ s	14.7 μ s/ 46.0×	4.0 μ s/12.6×
mbedtls	AES-256	0.16 μ s	26.8 μ s/143.3×	5.9 μ s/37.4×
OpenSSL	AES-256	0.16 μ s	24.6 μ s/156.3×	6.5 μ s/40.9×
Nettle	AES-256	0.22 μ s	22.8 μ s/103.3×	5.7 μ s/25.7×
LibTomCrypt	AES-256	0.89 μ s	23.5 μ s/ 26.5×	6.8 μ s/ 7.6×

Table 3: Runtime Performance Evaluation on Intel i9-10900K

5.4 Comparison to Related Methods

We compare our new framework to *RiCaSi*, the only framework that generates hardened binaries that are statically verified to be secure against cache side-channels. For the comparison, we apply *RiCaSi* to the same block cipher implementations as HyCaMi. Table 5 shows the comparison of runtimes for the combined hardened key schedule and encryption implementation. Since *RiCaSi* only offers the option to completely harden a program, the comparison focuses on the fully secure SecLUT-6 configuration. Our comparison shows that HyCaMi is up to 9.6× faster than *RiCaSi*. Moreover, we can also see that the speedup is higher for Camellia, and the four AES-256

	Library	Cipher	Attacker Model			
			<i>acc</i>	<i>accd</i>	<i>trace</i>	<i>time</i>
Original	mbedTLS	DES	32.6	32.0	120.0	6.9
	mbedTLS	3DES	32.6	32.0	376.0	8.5
	mbedTLS	Camellia	17.8	16.0	236.0	7.9
	mbedTLS	AES-256	71.0	66.3	275.0	8.1
	OpenSSL	AES-256	68.1	64.5	274.0	8.1
	Nettle	AES-256	71.6	66.3	271.0	8.0
	LibTomCrypt	AES-256	161.3	99.3	274.0	8.1
SecLUT-6	mbedTLS	DES	0.0	0.0	0.0	0.0
	mbedTLS	3DES	0.0	0.0	0.0	0.0
	mbedTLS	Camellia	0.0	0.0	0.0	0.0
	mbedTLS	AES-256	0.0	0.0	0.0	0.0
	OpenSSL	AES-256	0.0	0.0	0.0	0.0
	Nettle	AES-256	0.0	0.0	0.0	0.0
	LibTomCrypt	AES-256	0.0	0.0	0.0	0.0
SecLUT-8	mbedTLS	DES	2.0	0.0	14.0	3.9
	mbedTLS	3DES	2.0	0.0	28.0	4.9
	mbedTLS	Camellia	4.0	0.0	58.0	5.9
	mbedTLS	AES-256	13.4	2.6	143.0	7.2
	OpenSSL	AES-256	9.3	1.0	0.0	0.0
	Nettle	AES-256	13.4	2.6	135.0	7.0
	LibTomCrypt	AES-256	9.7	2.0	88.0	6.5

Table 4: Cache Side-Channel Leakage Bounds in [bit] for a 32KiB, 8-way associative, cache with 64 byte cache lines and LRU policy

Library	Cipher	HyCaMi	RiCaSi [21]	Speedup
mbedTLS	DES	6.0 μ s	7.0 μ s	1.2×
mbedTLS	3DES	17.8 μ s	26.8 μ s	1.5×
mbedTLS	Camellia	14.7 μ s	103.5 μ s	7.0×
mbedTLS	AES-256	26.8 μ s	216.8 μ s	8.1×
OpenSSL	AES-256	24.6 μ s	214.9 μ s	8.7×
Nettle	AES-256	22.8 μ s	217.9 μ s	9.6×
LibTomCrypt	AES-256	23.5 μ s	225.7 μ s	9.6×

Table 5: Comparing the runtime of key schedule and encryption between ciphers hardened with HyCaMi (SecLUT-6) and RiCaSi [21]

implementations. That is, our implementation is more effective for the four block ciphers with larger LUTs in the initial program.

6 CONCLUSION

This paper introduces HyCaMi, a pioneering framework designed for enhancing the security of block cipher implementations against cache side-channel attacks. This advancement leverages LUT-based high-level synthesis combined with quantitative side-channel analysis. Central to our methodology is a novel technique for translating LUT-based circuits into C code. This allows us to delve into the trade-offs between side-channel security and operational efficiency by varying the LUT sizes. Our implementation of HyCaMi yields two distinct outcomes: one is a 6-input LUT-based binary inherently immune to cache side-channel attacks, and the other is an 8-input LUT-based binary that maintains minimal leakage potential.

ACKNOWLEDGMENTS

This project was funded by the DFG within SFB 1119 CROSS-ING/236615297 and GRK 2050 Privacy & Trust/251805230, and by the ERC under the EU's Horizon 2020 program (grant No. 850990 PSOTI).

REFERENCES

- [1] 2020. XLS: Accelerated HW Synthesis. <https://google.github.io/xls>.
- [2] Lawrence E Bassham III. 2002. The advanced encryption standard algorithm validation suite (AESAVS). *NIST Information Technology Laboratory* (2002).
- [3] Daniel J. Bernstein. 2005. *Cache-timing attacks on AES*. Technical Report. University of Illinois at Chicago.
- [4] Guido Bertoni, Vittorio Zaccaria, Luca Breveglieri, Matteo Monchiero, and Gianluca Palermo. 2005. AES power attack based on induced cache miss and countermeasure. In *ITCC*.
- [5] Eli Biham. 1997. A fast new DES implementation in software. In *FSE*.
- [6] Pietro Borrello, Daniele Cono D'Elia, Leonardo Querzoni, and Cristiano Giuffrida. 2021. Constantine: Automatic side-channel resistance using efficient control and data flow linearization. In *CCS*.
- [7] Andreas Brüggemann, Robin Hundt, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2023. FLUTE: Fast and Secure Lookup Table Evaluations. In *S&P*.
- [8] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *ACM SIGACT-SIGPLAN*.
- [9] Lesly-Ann Daniel, Sébastien Bardin, and Tamara Rezk. 2020. Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary-Level. In *S&P*.
- [10] Yann Disser, Daniel Günther, Thomas Schneider, Maximilian Stillger, Arthur Wigandt, and Hossein Yalame. 2023. Breaking the Size Barrier: Universal Circuits meet Lookup Tables. In *ASIACRYPT*.
- [11] Goran Doychev and Boris Köpf. 2017. Rigorous Analysis of Software Countermeasures against Cache Attacks. In *ACM PLDI*.
- [12] Goran Doychev, Boris Köpf, Laurent Mauborgne, and Jan Reineke. 2015. Cacheaudit: A tool for the static analysis of cache side channels. *ACM TISSEC* 18, 1 (2015).
- [13] OpenSSL Software Foundation. 2023. *OpenSSL (Version 1.0.1d)*.
- [14] David Gullasch, Endre Bangerter, and Stephan Krenn. 2011. Cache games—bringing access-based cache attacks on AES to practice. In *IEEE S&P*.
- [15] Jonathan Heusser and Pasquale Malacaria. 2010. Quantifying Information Leaks in Software. In *ACM ACSAC*.
- [16] Ke Jiang, Yuyan Bao, Shuai Wang, Zhibo Liu, and Tianwei Zhang. 2022. Cache Refinement Type for Side-Channel Detection of Cryptographic Software. In *CCS*.
- [17] Emilia Käsper and Peter Schwabe. 2009. Faster and timing-attack resistant AES-GCM. In *CHES*.
- [18] libtom projects. 2018. *LibTomCrypt (Version 1.18.2)*.
- [19] ARM Limited. 2023. *mbedTLS (Version 2.16.5)*.
- [20] Pasquale Malacaria, MHR. Khouzani, Corina S. Păsăreanu, Quoc-Sang Phan, and Kasper S. Luckow. 2018. Symbolic Side-Channel Analysis for Probabilistic Programs. In *IEEE CSF*.
- [21] Heiko Mantel, Lukas Scheidel, Thomas Schneider, Alexandra Weber, Christian Weinert, and Tim Weißmantel. 2020. RiCaSi: Rigorous Cache Side Channel Mitigation via Circuit Compilation. In *CANS*.
- [22] Heiko Mantel, Alexandra Weber, and Boris Köpf. 2017. A Systematic Study of Cache Side Channels across AES Implementations. In *ESSOS*.
- [23] Niels Möller. 2019. *Nettle (Version 3.5)*. <https://ftp.gnu.org/gnu/nettle/nettle-3.5.tar.gz>
- [24] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2021. SynCirc: Efficient Synthesis of Depth-Optimized Circuits for Secure Computation. In *HOST*.
- [25] Ashay Rane, Calvin Lin, and Mohit Tiwari. 2015. Raccoon: Closing Digital Side-Channels through Obfuscated Execution. In *USENIX Security*.
- [26] Geoffrey Smith. 2009. On the foundations of quantitative information flow. In *FoSSaCS*.
- [27] Yukiyasu Tsunoo, Teruo Saito, Tomoyasu Suzuki, Maki Shigeri, and Hiroshi Miyauchi. 2003. Cryptanalysis of DES implemented on computers with cache. In *CHES*.
- [28] Shuai Wang, Yuyan Bao, Xiao Liu, Pei Wang, Danfeng Zhang, and Dinghao Wu. 2019. Identifying Cache-Based Side Channels through Secret-Augmented Abstract Interpretation. In *USENIX Security*.
- [29] Alexandra Weber, Oleg Nikiforov, Alexander Sauer, Johannes Schickel, Gernot Alber, Heiko Mantel, and Thomas Walther. 2021. Cache-Side-Channel Quantification and Mitigation for Quantum Cryptography. In *ESORICS*.
- [30] Clifford Wolf. 2016. Yosys Open SYNthesis Suite. <https://yosyshq.net/yosys/>.
- [31] Meng Wu, Shengjian Guo, Patrick Schaumont, and Chao Wang. 2018. Eliminating Timing Side-Channel Leaks Using Program Repair. In *ACM ISSTA*.
- [32] Yuval Yarom and Katrina Falkner. 2014. FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack. In *USENIX Security*.

An Efficient Hardware Accelerator Design for Dynamic Graph Convolutional Network (DGCN) Inference

Yingnan Zhao*, Ke Wang[§], Jiaqi Yang*, Ahmed Louri*

* The George Washington University, Washington, D.C.

[§] University of North Carolina at Charlotte, Charlotte, NC

{yzhao96, Yang_Jiaqi_Cute, louri}@gwu.edu, ke.wang@charlotte.edu

ABSTRACT

Dynamic graph convolutional networks (DGCNs) have been increasingly used to extend machine learning techniques to applications that involve graph-structured data with temporal changes. A typical DGCN model is comprised of graph convolutional network (GCN) layers to capture spatial information, followed by recurrent network (RNN) layers for temporal information. Designing a high-performance and energy-efficient DGCN accelerator is challenging due to the distinct computation and communication requirements of the GCN and RNN layers. Specifically, the computation of GCN layers can be abstracted as Sparse-dense and General Matrix-matrix Multiplication (SpMM and GeMM), while RNN layers involve extensive element-wise addition and Hadamard product in addition to SpMM and GeMM. For data communication, GCN layers necessitate irregular data memory access due to the unstructured distribution of vertices involved in graphs, whereas RNN layers exhibit a predictable memory access pattern. We propose E-DGCN, a high-performance and energy-efficient accelerator design for improved DGCN inference. The proposed E-DGCN comprises reconfigurable processing elements that efficiently support diverse types of data computations required by GCN and RNN layers, a flexible on-chip interconnection design with an adaptive dataflow to improve data reuse during DGCN inference, and a lightweight vertex caching algorithm to leverage data locality and reduce off-chip memory access while processing temporal information. Experimental results show that the E-DGCN achieves 2.2x speed-up and 2.6x energy savings on average as compared to existing DGCN accelerators.

1 INTRODUCTION

Dynamic graphs are pervasive data structures that model pairwise interactions between entities in systems that are constantly changing [1–4]. To extract spatial-temporal features, dynamic graph convolutional networks (DGCNs) have been developed to facilitate machine learning on dynamic graphs applied in a wide variety of application domains, such as social networks [5], recommendation systems, traffic forecasting [6], and many others. A typical DGCN model, as shown in Fig. 1, is comprised of two types of neural network layers, namely the graph convolutional network (GCN)

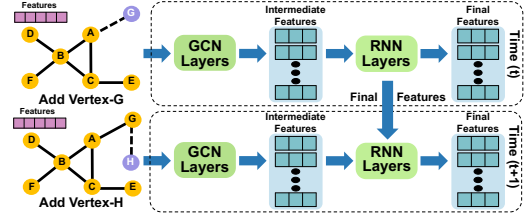


Figure 1: A typical DGCN model includes graph convolutional network (GCN) layers to capture spatial evolution and recurrent neural network (RNN) layers to capture temporal information of input graphs.

layers to capture spatial dependencies and perform conventional static graph learning [7, 8], and the recurrent neural network (RNN) layers to process temporal information [1, 2, 6] that captures the dynamic change of graph representations.

In DGCN models, GCN and RNN layers have distinct data computation and communication requirements. Specifically, for computation, each GCN layer takes the adjacency, feature, and weight matrices as input and performs Sparse-dense and General matrix-matrix multiplications (SpMM and GeMM). For each RNN layer, it takes the results of previous GCN and RNN output as input matrices to be multiplied by weight matrices to perform SpMM. Additionally, the intermediate matrices of each RNN layer are used to calculate Hadamard Product [9]. For data communication, GCN layers induce irregular data memory access and unpredictable data reuse, due to the varying numbers and locations of neighboring nodes of each vertex in the input graphs. On the contrary, since RNN layers recurrently use the output matrices of GCN layers whose vertices share the same weight matrices, it results in a predictable data reuse pattern and regular data memory access. Therefore, there is a strong need for a versatile design that can dynamically support diverse computation types required by RNN and GCN layers, along with a flexible dataflow to efficiently handle memory access, explore data locality, and support diverse data reuse patterns.

In this paper, we propose E-DGCN, an efficient architecture design aimed at accelerating DGCN inference with improved performance and reduced energy consumption. The key contributions are abstracted as follows:

- E-DGCN features a unified computing array that includes multiple reconfigurable Processing Elements (PEs). Each PE can dynamically adjust its functionality to efficiently support various computations required by GCN and RNN layers, including SpMM, GeMM, element-addition, Hadamard product, and diverse activation functions.
- E-DGCN deploys an adaptive on-chip dataflow facilitated by a flexible interconnection design. According to GCN and RNN workloads, the flexible interconnection enables

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

<https://doi.org/10.1145/3649329.3658254>

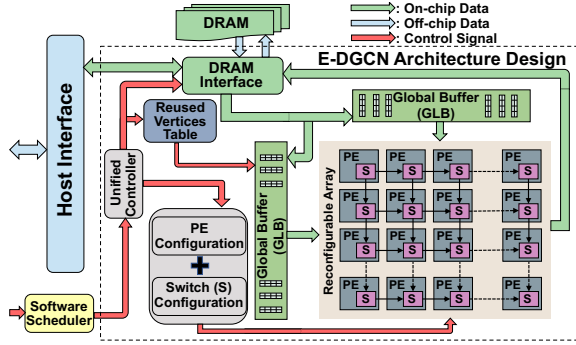


Figure 2: The overall architecture of the proposed design includes Processing Elements (PEs) and Switches (S) used for on-chip data computation and communication, respectively. The Global Buffer (GLB) stores the required data loaded from DRAM. The Unified Controller, connected to the software scheduler, selects the configuration of PEs and switches based on the input workload. The Reused Vertices Table records the indices of vertices that can be further reused, as determined by the proposed algorithm.

the most efficient dataflow (i.e., inter-PE data transmission, spatial-temporal decision, and on-chip buffering strategy) that explores data locality to improve data reuse among PEs.

- E-DGCN integrates a lightweight vertex caching algorithm that exploits the intra-layer and inter-layer data dependencies to locate the dynamic change of the graph (e.g., addition and removal of nodes/edges). Using the proposed algorithm, E-DGCN selectively loads the vertices impacted by the temporal change from the main memory to the accelerator during DGCN inference, which significantly contributes to reducing off-chip data memory access.

Evaluation results with real-world datasets show that the proposed E-DGCN achieves a factor of 2.2x speed-up and 2.6x energy savings on average as compared to previous designs.

2 PROPOSED E-DGCN DESIGN

2.1 Architecture Overview

Fig. 2 demonstrates the overall architecture of the proposed E-DGCN. E-DGCN implements a tile-based unified reconfigurable array that includes customized Processing Elements (PE) and an interconnection design with reconfigurable switches (S) to fulfill the diverse data computation and communication requirements that exist in the DGCN model, respectively. Specifically, based on the input workload, the proposed PE can adjust its functionality to perform various types of computations, including matrix-matrix multiplication, Hadamard product, and activation functions. Each PE is connected to its local reconfigurable switch, and switches are interconnected to support an adaptive dataflow and manage the on-chip data communication efficiently. Details of the proposed E-DGCN PEs and switches are introduced in Sec. 2.2 and Sec. 2.3, respectively. The Global Buffer (GLB) is implemented as a multi-bank scratchpad memory, where each bank can be shared by different PEs through E-DGCN switches. Additionally, E-DGCN features a unified controller connected to the off-chip software scheduler. In response to the input workload from the software scheduler, the unified controller selects the configuration for both E-DGCN PEs

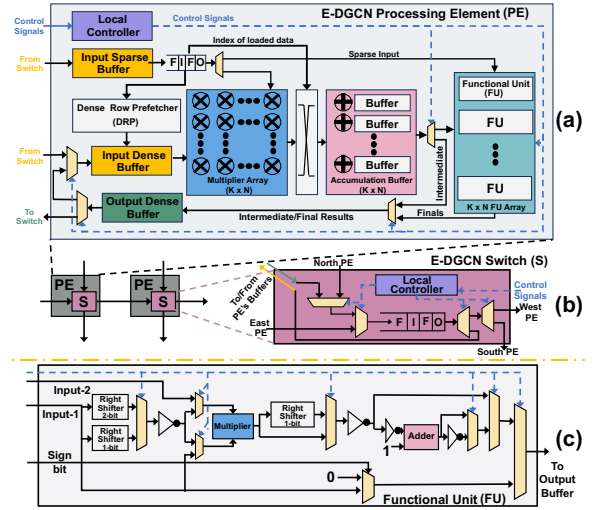


Figure 3: Architecture design of the proposed E-DGCN processing elements (PEs). The multiplier array followed by accumulation buffers, is implemented for Sparse-dense, General matrix-matrix multiplication (SpMM or GeMM), and element-wise addition. (b) Architecture design of the proposed switch (S) for managing on-chip data communication. (c) Functional units (FUs) perform Hadamard Product and activation functions for both GCN and RNN layers. Black and blue links represent data and control signal transmission.

and switches, forwarding the corresponding control signals to the entire reconfigurable array. To efficiently manage data communication between E-DGCN and DRAM while avoiding repeated access to the same set of vertices, a table is utilized to record indices of reused vertices that is directed by a proposed lightweight vertex caching algorithm which is detailed in Sec. 2.4.

2.2 E-DGCN Processing Element (PE)

Fig. 3 (a) depicts details of the proposed E-DGCN Processing Element (PE). Specifically, considering that the input matrices for both GCN and RNN layers consist of both sparse and dense matrices stored in different formats, each modified PE incorporates separate input sparse and dense buffers to store input data separately for improved storage efficiency. To facilitate data synchronization during computations, when the sparse input data is streamed into the First-In-First-Out (FIFO) from the input sparse buffer, the indexes of non-zero elements are sent to the Dense Row Prefetcher. Subsequently, these indexes are forwarded to the input dense buffer to load the specific row. Additionally, each PE integrates a dense output buffer to store the intermediate results or the final outputs of the RNN and GCN layers. The local controller receives the control signal based on the input workload and configures MUX-DeMUXes to manage the local communication. To perform diverse types of computations, each PE consists of three main hardware components: a multiplier array, an accumulation buffer array, and a versatile functional unit (FU) array.

For GCN layers: Each GCN layer takes the normalized adjacency matrix ($\hat{A}$), feature matrix (X), and weight matrix (W) as inputs to calculate the final output $X^{(K+1)} = \sigma(\hat{A}X^{(K)}W^{(K)})$, where K represents the number of GCN layer. The computation consists

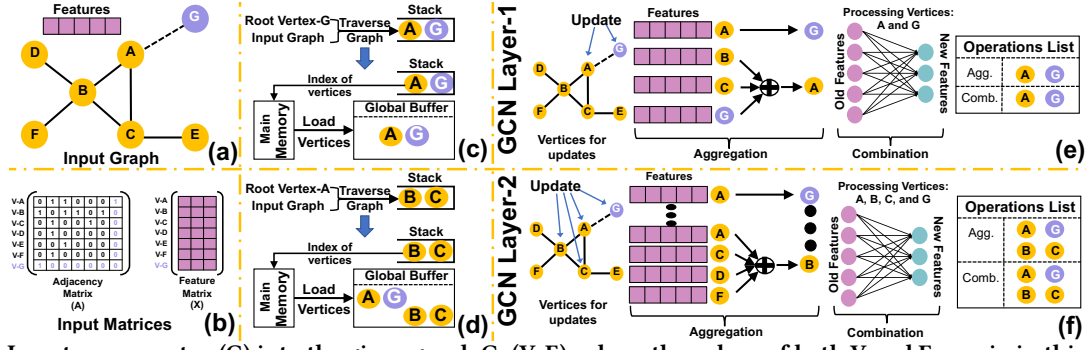


Figure 4: (a) Insert a new vertex (G) into the given graph $G=(V, E)$, where the values of both V and E are six in this example. (b) The update of both adjacency and feature matrices after inserting vertex-G. Note that in real applications, the adjacency matrix is typically normalized. In this example, we set values of the adjacency matrix to 0 or 1 for simplicity. (c) and (d) illustrate the evolution of loaded vertices stored in the global buffer for two GCN layers, respectively. The stack is used to record the datapath during graph traversal. Additionally, (e) and (f) depict the processing of aggregation and combination phases of loaded vertices, providing a detailed list of the operations that occurred.

of an aggregation phase (SpMM) and a combination phase (GeMM). In this case, each PE activates the unified multiplier array followed by accumulation buffers to perform the required computations. Those multipliers located within the same row/column share the same data from the input dense/sparse buffers, respectively. Partial sums of multipliers are then forwarded to the accumulation buffers through a permutation network for further addition. The permutation network is controlled by the index of data stored within the FIFO and is used to pair the multipliers with the corresponding accumulation buffers. Additionally, each PE applies the Outer Product (OP) loop unrolling technique when performing SpMM and GeMM. Compared to the other two common loop unrolling techniques, namely Inner Product (IP) [10] and Row-based Product (RP), OP can mitigate the impact of the workload imbalance induced by the sparsity of the graph's structure [11–14]. Each functional unit, as shown in Fig. 3 (c), is configured to perform activation functions, such as ReLU and Sigmoid.

For RNN layers: Each RNN layer utilizes the output of previous GCN and RNN layers as inputs ($x(t)$ and $h(t-1)$) multiplied by weight matrices to produce the final result. The computation can be abstracted as $h(t) = f(U \times x(t) + W \times h(t-1))$, where U and W represent the weight matrices, and f is a function that calculates the Hadamard product. Similarly to the GCN layer, the SpMM in the RNN layer is performed by the multiplier array and the accumulation buffers. Additionally, the accumulation buffers also perform element-wise addition on the matrix level with intermediate results of RNN layers. Each FU serves two functionalities: (1) calculating the Hadamard products, and (2) performing activation functions, including ReLU, Sigmoid, and Tanh.

2.3 Flexible Interconnection Design

To effectively manage data communication among PEs and support the adaptive dataflow, the proposed design integrates a flexible interconnection design with reconfigurable switches, as shown in Fig. 3 (b). Each switch features two input ports, two output ports, a local port, and a First-In-First-Out (FIFO) buffer. The input/output ports are used to share data communication with adjacency switches. The FIFO is used to store the data received from the corresponding PE or neighboring switches. Additionally, several MUX/DEMUXes are

used to adjust the directions of data transmission, which are configured by the local controller according to the required dataflow. To ensure data synchronization and eliminate per-hop control timing overhead, those switches within the same row or column work in a systolic manner, wherein all the tiles are simultaneously sent across the PE array. The working of the proposed interconnection network and supported dataflows for the GCN and RNN layers are detailed as follows.

For GCN layers: Given that E-DGCN follows a tile-based architecture, both adjacency and feature matrices are partitioned into compact tiles to align with on-chip storage constraints. Each PE handles the computation of a specific pair of tiles. During the **aggregation phase**, to maintain data synchronization, PEs in the same row are assigned to the same set of vertices, while those PEs in the same column manage identical sets of feature vectors. Additionally, each simplified switch configures local MUX-DeMUXes to establish links with neighboring switches in the same row, and each E-DGCN PE is set up to execute Sparse-dense/General Matrix Multiplication (SpMM/GeMM) computations. During the **combination phase**, the horizontal connections remain active. Given that the weight matrix of the GCN layer is small and dense [13], each PE stores the weight matrix directly in its local input dense buffer without sharing. Once the combination phase is completed, different features of the same vertex in those PEs in the same row are streamed to the GLB and then forwarded to the DRAM.

For RNN layers: Each RNN layer includes multiple gate functions and each gate function has two types of input matrices: one is the result of the current GCN layer (A) and the other is the result of the latest RNN layer (B). Each gate function utilizes two distinct weight matrices to separately multiply two types of input and perform the element addition at the end [1, 2]. Therefore, the two types of input matrices are shared among diverse gate functions. Under this circumstance, E-DGCN assigns the entire row of PEs to the same set of vertices and the entire column of PEs to the same type of weight matrix. During the computations, the tiles of the input matrix are shared by PEs horizontally. After finishing the transmission of the input B matrix, the A matrix will be streamed in the same way as the B matrix. Concurrently, the result of each PE stays locally for accumulation. When completing both types of

Algorithm 1 Light-weight Vertex Caching Algorithm

```
1: Inputs: The number of vertices (V), the graph structure (adjacency_matrix), the root vertex (R), the array includes the id of vertices located in global buffer (array_reuse_vertices[]), and the layer number of the ongoing GCN model (n)
2: Outputs: The array that includes the required vertices to be loaded from the main memory.
3: Begin:
4: // record the visit status of vertices.
5: boolean visited[V]
6: level = 0 // record the height of the DFS tree
7: // Initialize the status of all vertices to un-visited
8: for v in V do
9:   visited[v] = false
10: end for
11: stack(G) // the datapath for recording traversal
12: // A DFS-based recursive traverse function
13: traverse(R, adjacency_List[R], level)
14: // record the vertices needed from the main memory
15: array = stack() - array_reuse_vertices[]
16: return array
```

input matrices, vertical links are active while horizontal links are disabled for result accumulation.

2.4 Lightweight Vertex Caching Algorithm

Dynamic graphs encompass various operations involving the addition or removal of edges, features, and vertices over time [3, 15]. These operations impact DGCN inference as changes occur in the input adjacency and feature matrices. Fig. 4 shows an example of the addition of vertices in DGCN, with other operations being abstracted similarly. We utilize a DGCN model that includes two conventional GCN layers in this example. As shown in Fig. 4 (a), the system inserts a new vertex (G) to the given graph $G=(V, E)$, where V and E represent the number of vertices and edges, respectively (both V and E are six in this example). By adding the new vertex, modifications occur in both the adjacency and feature matrices, as illustrated in Fig. 4 (b). In this scenario, only vertices A and G undergo the aggregation and combination phases of GCN Layer-1. This is because the addition of vertex-G specifically influences the arrangement of vertex-A's neighbors. Following the completion of Layer-1, the updated features of vertices A and G necessitate updates in the second GCN layer for vertices A, B, C, and G, as illustrated in Fig. 4 (d). This is because the update of vertex A has an impact on both vertices B and C. With two GCN layers, the operation of inserting a new vertex G only impacts the results of vertices A, B, and C, which is limited for DGCN inference. Additionally, throughout the entire GCN inference, the values of the adjacency list and intermediate results of vertices A and G can be reused.

Based on the aforementioned observation, E-DGCN deploys a lightweight vertex caching algorithm based on Deep-First Search (DFS) to efficiently locate those vertices impacted by the update, efficiently manage, and avoid redundant data memory access, as illustrated in Algorithm-1. Specifically, the proposed algorithm designates the updated vertex (N) as the root node and loads the adjacency list of the vertex N to an array. Subsequently, the proposed algorithm calls a recursive traverse function, as shown in Algorithm-2, which takes the root node and the adjacency list as inputs to traverse the graph, identifying those vertices impacted by

Algorithm 2 Recursive Traverse Function

```
1: Inputs: Root vertex (R), adjacency List of vertex-R (adjacency_List[R]), the degree of the DFS tree (level)
2: Begin:
3: void traverse (Vertex R, Adjacency_List adjacency_List[R], int level)
4: for v in adjacency_List[R] do
5:   if visited[v] then
6:     // v is already in the stack
7:   else
8:     stack(v)
9:     visited(v) = true
10:   end if
11:   if level == n then
12:     // Begin backtrace, and the recursion is finished.
13:   else
14:     traverse (v, adjacency_List[v], level+1)
15:   end if
16: end for
```

the updated vertex N. The termination condition for the recursive function is based on the height of the search tree. The maximum value of the height is determined by the total number of GCN layers involved in DGCN models. After the traverse function is completed, the system receives an array (stack), including vertices that need to be loaded for the current GCN layer. As the number of processing GCN layers increases, the height of the traversal tree also grows. Moreover, vertices at the top of the tree are frequently accessed. To avoid redundant off-chip memory access of these vertices, the proposed design records the index of these vertices using an additional array and stores the intermediate results of these vertices on-chip for subsequent reuse, thus reducing data memory access. Given a DGCN model with N GCN layers, the time complexity of the proposed algorithm is $O(V+E)$, where V and E represent the number of vertices and edges traversed by the algorithm.

An example of the proposed vertex caching algorithm is demonstrated in Fig. 4. As shown in Fig. 4, the proposed algorithm designates the inserted vertex G as the root node. Subsequently, the system traverses all its neighbors (A in this example) and stacks them for the first GCN layer. Fig. 4 (c) and (e) illustrate the vertices stored inside the global buffer and the detailed operations performed on the loaded vertices for the first GCN layer, respectively. After completing the aggregation and combination phases for all the loaded vertices, the proposed algorithm uses these loaded vertices as root nodes to explore new required vertices at a deeper level through the traverse function. Simultaneously, the results of both vertex A and G from the first GCN layer are stored within the on-chip buffer for future data reuse. The index and the height of A and G are stored in the on-chip table for easy lookup. Fig. 4 (d) and (f) illustrate the vertices stored inside the global buffer and the detailed operations performed on the loaded vertices for the Layer-2 of GCN inference, respectively.

3 EVALUATION AND ANALYSIS

3.1 Experiment Setup

Hardware simulator. We build a cycle-accurate simulator in C++ language to model the hardware behavior and evaluate the performance of the proposed design. Specifically, the designed simulator

Table 1: Details of datasets used for evaluation

Datasets	Vertices	Edges	Features	Description
Wikipedia (WI)	9,227	157,474	172	Citation Graph
Reddit (RD)	55,863	858,490	602	Social Graph
Twitter (TW)	8,861	119,872	768	Sharing Graph
PubMed (PM)	1,917	88,648	500	Citation Graph

precisely counts the exact number of on/off-chip data memory read and write operations, which is used to estimate the energy consumption of the memory access according to [16]. To measure the area consumption, we model all the proposed hardware logic, including the unified reconfigurable array, controller, MUX-DeMUXes, FIFOs, and other hardware components. We use the Synopsys Design Compiler with the TSMC 45nm library for the synthesis. We set the clock frequency at 1 GHz. We use Cacti 6.0 [17] to estimate the area, power, and access latency of all types of on-chip buffers.

Architecture Configuration. We implement the proposed design including K×K PEs/simplified routers array. Each PE includes an N×N multiplier array connected to an accumulation buffer with N×N adders. Additionally, each PE includes N×N functional units for required activation functions. During the evaluation, we set the values of K and N to be 8 and 4, respectively. Additionally, in Sec. 3.5, we conduct a scalability analysis to illustrate the relationship between performance and the dimensions of the unified array. Since the proposed E-DGCN is a tile-based architecture, the capacity of the input sparse/dense buffer within PEs is 320 KB and 4 KB to accommodate the required data of each tile. The output buffer used to store the intermediate and final matrices is 256 KB. The capacity of the FIFO inside each simplified router is 320 KB, designed to accommodate the largest tile of data that needs to be transferred between modified PEs such as feature matrices.

Datasets and Benchmarks. Table 1 illustrates four dynamic graphs used for evaluation in this paper with the number of vertices and edges [1, 18, 19]. We consider two typical DGCN models: T-GCN [6] and CD-GCN [1]. Specifically, T-GCN includes GCN and Gated Recurrent Unit (GRU). CD-GCN includes GCN and Long-Short-Term-Memory (LSTM) models. The 32-bit floating-point representation is used in the evaluation, which proves to be sufficient for maintaining inference accuracy [20, 21].

Baselines. To perform fair cross-platform comparisons, we compare E-DGCN to the state-of-the-art DGCN software framework, PyGT [22], on CPU (Intel Xeon V4) and GPU (NVIDIA A100), identified as PyGT-CPU and PyGT-GPU, respectively. We also compare E-DGCN to prior work on DGCNs such as ReaDy [2]. Additionally, since DGCNs mainly include two basic kernels, GCN and RNN, we compare E-DGCN to the combination of state-of-the-art GCN (I-GCN [12]) and RNN (ERA-LSTM [23]) accelerators, connecting as a tandem-engine architecture (denoted as I-LSTM).

3.2 Data Memory Access

Fig. 5 shows the normalized off-chip data memory access of the proposed design (E-DGCN) compared to previous works. All the evaluation results are normalized to the proposed design, and a lower value indicates better performance. E-DGCN outperforms

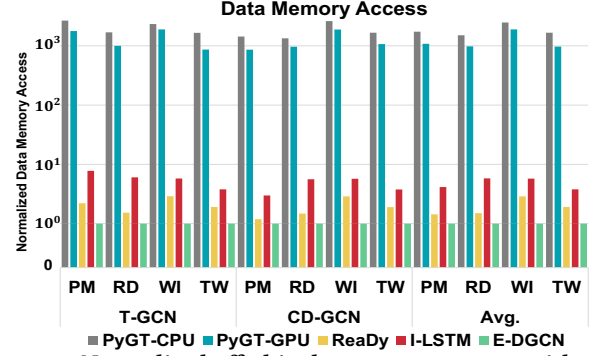


Figure 5: Normalized off-chip data memory access with two different DGCN models on four datasets (lower is better).

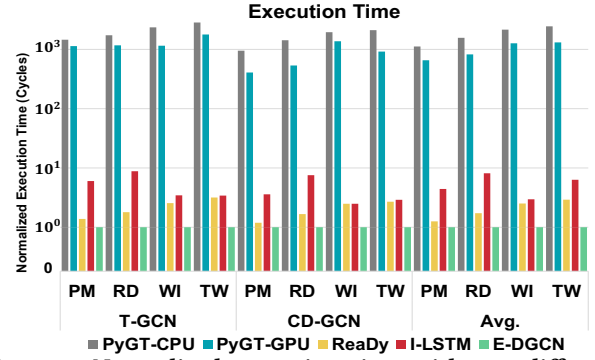


Figure 6: Normalized execution time with two different DGCN models on four datasets. All the data is evaluated in cycles (lower is better).

previous approaches for several reasons. Firstly, compared to conventional processing units (CPU and GPU), E-DGCN can directly utilize the sparse input matrices stored in a compressed format without under-utilizing hardware resources, such as on-chip data storage. In comparison to ReaDy, E-DGCN implements a dynamic dataflow when performing different neural network models to maximize on-chip data reuse, thereby reducing off-chip memory access. Additionally, benefiting from the proposed algorithm, E-DGCN can selectively load required vertices during each GCN layer instead of loading the entire graph. This approach minimizes unnecessary data memory access induced by unimpacted vertices when operations happen, such as when inserting or removing vertices from the graph. Moreover, the proposed algorithm allows E-DGCN to effectively store the intermediate results of inter-layers of the GCN model locally instead of writing them back to the main memory. All methodologies mentioned above help the proposed design reduce off-chip data memory access by nearly 50% on average compared to the ReaDy design.

3.3 Execution Time

Fig. 6 illustrates the execution time of the proposed design in comparison to previous works, measured in terms of the total number of execution cycles. The execution time includes the duration of configuration calculations, the transmission of relevant control signals, and the setup of the entire architecture. E-DGCN achieves an average of 54.5% reduced execution time ($2.2 \times$ speedup) compared to

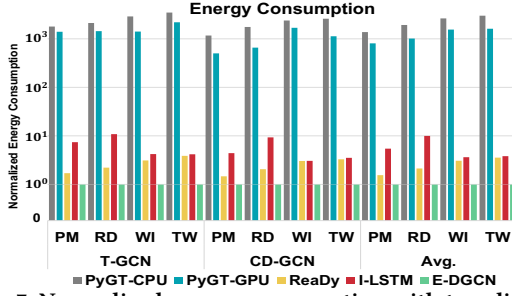


Figure 7: Normalized energy consumption with two different DGCN models on four datasets (lower is better).

previous works. This is because E-DGCN provides improved computation and communication. Specifically, compared to crossbar-based approaches, the proposed design utilizes the sparse data format directly, eliminating the time required for data format conversion from sparse to dense. Furthermore, compared to the tandem-engine design where two engines may idle while waiting for results from each other, the unified reconfigurable array helps minimize idle time, leading to improved performance. Furthermore, the flexible interconnection design and vertex caching algorithm contribute to performance improvement with reduced off-chip data memory access and fine-tuned dataflow.

3.4 Energy Consumption

All approaches estimate the related energy consumption according to [17], and are normalized based on the proposed design. As shown in Fig. 7, the proposed design achieves around 61.5% (2.6 $\times$) energy savings on average compared to previous customized designs. This is because our proposed architecture design has reduced DRAM accesses, improved on-chip data reuse, and efficient execution of data computations.

3.5 Scalability Analysis

We evaluate the scalability of the proposed architecture by varying the dimension of the unified reconfigurable array, using the T-GCN model with the Twitter dataset. The array size is scaled from 1 $\times$ 1 to 8 $\times$ 8. All performance metrics are measured in cycles and normalized to the performance of the 1 $\times$ 1 dimension. As depicted in Fig. 8, even though the scaling of PE array size continues to induce execution time reduction, the improvement is diminished. This is because the main memory bandwidth becomes the primary performance bottleneck as the dimension of the PE array increases, especially when substantial computing resources are available for the given DGCN models in use, according to the Roofline model [24].

3.6 Area Consumption

We evaluate the area consumption of the proposed architecture under TSMC 45 nm technology, and E-DGCN occupies a total area of 45.58 (mm^2). Specifically, for the proposed PE, the three types of input buffer consume around 68.6% of the total PE area, while the switch with embedding a FIFO (320 KB) consumes around 30.5%. In the overall architecture design, the 4 $\times$ 4 unified reconfigurable array occupies the majority of the area, accounting for approximately 72.3%. The reused vertices table consumes 0.2% chip area.

4 CONCLUSION

We propose an efficient accelerator design named E-DGCN for high-performance and energy-efficient Dynamic Graph Convolutional

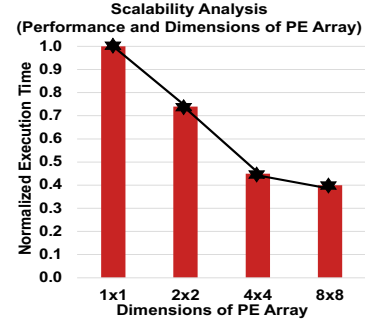


Figure 8: Normalized execution time when scaling the dimensions of the unified reconfigurable array (lower is better).

Network (DGCN) inference. Specifically, E-DGCN includes reconfigurable Processing Elements (PEs) with a flexible interconnection design to support diverse types of data computations and various dataflows, with maximized on-chip data reuse and hardware utilization. Additionally, a lightweight vertex caching algorithm is proposed to exploit data locality, enabling E-DGCN to selectively load required vertices from the main memory during DGCN inference, thereby reducing off-chip data memory access.

REFERENCES

- [1] Aldo Pareja et al. Evolvegcn: Evolving graph convolutional networks for dynamic graphs. In *Proc. of AAAI'20*, 2020.
- [2] Yu Huang et al. Ready: A reram-based processing-in-memory accelerator for dynamic graph convolutional networks. In *TCAD*, 2022.
- [3] Mahbod Afarin et al. Commongraph: Graph analytics on evolving data. In *Proc. of ASPLOS'23*, 2023.
- [4] Emanuele Rossi et al. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637*, 2020.
- [5] Avery Ching et al. One trillion edges: Graph processing at facebook-scale. *Proc. of the VLDB Endowment*, 2015.
- [6] Ling Zhao et al. T-gcn: A temporal graph convolutional network for traffic prediction. In *ITS*, 2019.
- [7] Justin Gilmer et al. Neural message passing for quantum chemistry. In *Proc. of ICML'17*, 2017.
- [8] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [9] Roger A Horn. The hadamard product. In *Proc. Symp. Appl. Math.*, 1990.
- [10] Shmuel Winograd. A new algorithm for inner product. In *TC*, 1968.
- [11] Tong Geng et al. Awb-gcn: A graph convolutional network accelerator with runtime workload rebalancing. In *Proc. of MICRO'21*, 2020.
- [12] Tong Geng et al. I-gcn: A graph convolutional network accelerator with runtime locality enhancement through islandization. In *Proc. of MICRO'21*, 2021.
- [13] Jiajun Li et al. Gcnax: A flexible and energy-efficient accelerator for graph convolutional neural networks. In *Proc. of HPCA'21*, 2021.
- [14] Jiajun Li et al. Sgcnax: A scalable graph convolutional neural network accelerator with workload balancing. In *TPDS*, 2021.
- [15] Maciej Besta et al. Practice of streaming processing of dynamic graphs: Concepts, models, and systems. In *TPDS*, 2021.
- [16] Song Han and Others. Eie: Efficient inference engine on compressed deep neural network. *ACM SIGARCH Computer Architecture News*, 2016.
- [17] Naveen Muralimanohar et al. Cacti 6.0: A tool to understand large caches. *University of Utah and Hewlett Packard Laboratories, Tech. Rep.*, 2009.
- [18] Palash Goyal et al. Dyngem: Deep embedding method for dynamic graphs. *arXiv preprint arXiv:1805.11273*, 2018.
- [19] Ryan Rossi and Nesreen Ahmed. The network data repository with interactive graph analytics and visualization. In *Proc. of AAAI'15*, 2015.
- [20] Seyed Mehran Kazemi et al. Representation learning for dynamic graphs: A survey. *The Journal of Machine Learning Research*, 2020.
- [21] Mingyu Yan et al. Hygcn: A gcn accelerator with hybrid architecture. In *Proc. of HPCA'20*, 2020.
- [22] Benedek Rozemberczki et al. Pytorch geometric temporal: Spatiotemporal signal processing with neural machine learning models. In *Proc. of CIKM'21*, 2021.
- [23] Jianhui Han et al. Era-lstm: An efficient reram-based architecture for long short-term memory. In *TPDS*, 2019.
- [24] Samuel Williams et al. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 2009.

FCM: A Fusion-aware Wire Cutting Approach for Measurement-based Quantum Computing

Zewei Mo, Yingheng Li, Aditya Pawar, Xulong Tang, Jun Yang, Youtao Zhang

University of Pittsburgh

Pittsburgh, Pennsylvania, USA

{zewei.mo,yil392,adp110,tax6,juy9}@pitt.edu,zhangyt@cs.pitt.edu

ABSTRACT

Measurement-based quantum computing (MBQC) is a promising quantum computing paradigm that carries out computation through *one-way* measurements on entangled photon qubits. Practical photonic hardware first generates a 2D mesh of resource states with each being a small number of entangled photon qubits and then exploits fusion operations to connect resource states to scale up the computation. Given that the fusion operation is highly error-prone, it is important to reduce the number of fusions for an MBQC circuit.

In this paper, we propose FCM, a fusion-aware scheme that exploits wire cutting to improve the fidelity of MBQC. By cutting a large MBQC circuit into several smaller subcircuits, FCM effectively reduces the number of fusions in each subcircuit and thus improves the computation fidelity. Given circuit cutting requires classical post-processing to combine the results of subcircuits, FCM strives to achieve the best cutting strategy under different settings. Evaluation of representative benchmarks demonstrates that, when cutting a large circuit to two subcircuits, FCM reduces the maximum number of fusions of all subcircuits by 59.6% on average (up to 69.1%).

CCS CONCEPTS

• Computer systems organization → Quantum computing.

KEYWORDS

Measurement-based Quantum Computing, Photonic, Fidelity

ACM Reference Format:

Zewei Mo, Yingheng Li, Aditya Pawar, Xulong Tang, Jun Yang, Youtao Zhang. 2024. FCM: A Fusion-aware Wire Cutting Approach for Measurement-based Quantum Computing. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3657346>

1 INTRODUCTION

Measurement-based quantum computing (MBQC) is a promising quantum computing paradigm that carries out computation through *one-way* measurements on entangled photon qubits [13]. Recent studies showed that practical photonic hardware first generates a 2D mesh of resource states with each being multiple entangled photon qubits and then exploits fusions to combine resource states to scale

up to accomplish computation tasks [16]. Compared to mainstream quantum computing paradigms (e.g., superconducting [12]), MBQC exhibits good scalability and long coherence time. Photonic chips that integrate both quantum photonic and electronic components are being manufactured in the industry [6].

However, fusion is a highly error-prone, low-fidelity quantum operation. Assuming its failure probability is α , and mapping a quantum circuit to photonic hardware results in n fusions, the circuit success rate is $(1 - \alpha)^n$, which deteriorates quickly with large n values. At $\alpha = 1\%$, an MBQC circuit having 69 or more fusions has a possibility lower than 50% to get the correct result. This clearly is a big concern nowadays as α is 10% [17] and a large quantum circuit may result in thousands of fusions. While the fusion failure is expected to reduce significantly with dramatic advancements in photonic technology in near-term [5], even at $\alpha=0.1\%$, we can only improve n to 693. Therefore, the number of fusion operations remains a major constraint in MBQC paradigm.

Zhang *et al.* propose a compilation framework *OneQ* [7] to map quantum circuits to physical resource states and take a pioneering step to reduce fusions from imperfect mapping. However, the number of fusions is mostly determined by the size and complexity of the quantum circuit. A large quantum circuit, even with *OneQ* optimization, may result in a physical mapping with more than an acceptable number of fusions. A simple yet effective approach is to cut a large quantum circuit into smaller subcircuits and exploit post-processing to combine the results from subcircuits, similar to that in wire cutting [18]. On the one hand, cutting improves fidelity as its fidelity is determined by the subcircuit with the most fusions. On the other hand, cutting introduces a post-processing overhead that increases exponentially with the number of cuts. Although previous studies have exploited wire cutting to decompose circuits [10, 18], they are fusion oblivious — since the number of fusions cannot be determined until the given circuit gets mapped to physical resource states. More importantly, existing wire-cutting-based solutions focus on generating subcircuits of limited width, which might result in an imbalanced number of fusions for each subcircuit, thereby suffering low overall fidelity.

In this paper, we propose FCM, a Fusion-aware Cutting approach for MBQC. FCM splits a large quantum circuit into multiple subcircuits such that each subcircuit has fewer fusion operations, which significantly improves the fidelity. We formulate the problem using mixed-integer programming (MIP) and exploit an MIP solver to optimize cutting decisions under different settings. When cutting a large circuit to m subcircuits, if our goal is to improve the fidelity, we strive to balance the number of fusion counts in all subcircuits; if our goal is to minimize the post-processing overhead, we prioritize the number of cuts in choosing the cutting positions. The balance

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06.

<https://doi.org/10.1145/3649329.3657346>

of fusion count per subcircuit would be relaxed as long as each subcircuit has fewer than a threshold of fusions.

To summarize, we make the following contributions.

- We reveal the potential opportunity of decomposing a circuit for fusion reduction through wire cutting.
- We formulate the problem using MIP and optimize cutting decisions under different settings.
- We evaluate FCM on representative benchmarks. Our results show that FCM achieves significant fusion reductions and post-processing overhead reductions.

2 BACKGROUND AND RELATED WORK

2.1 Measurement-based Quantum Computing

Measurement-based quantum computing (MBQC) is a promising quantum computing paradigm that carries out computation through *one-way* measurements on entangled photonic qubits [14]. When deploying a quantum circuit for MBQC by the compiler, several steps of transformation [7] are required to map the circuit to photonic hardware, which is demonstrated by Figure 1. A quantum circuit (Figure 1(a)) is first transformed into the graph state (Figure 1(b)). It is then transformed into the fusion graph (Figure 1(c)) by leveraging fusion operations. Each node in the fusion graph represents one resource state, e.g., a three-qubit resource state [11] in the figure. In the end, the fusion graph is mapped to the photonic hardware using the routing algorithm [7] (Figure 1(d)).

Graph state in MBQC. The graph state represents a quantum circuit as a graph of entangled qubits, i.e., $G = (V, E)$, where qubits at vertices are initialized as the $|+\rangle$ state and controlled-Z gates at edges are applied to the qubits whose corresponding vertices in the graph are connected. Thus, the graph state can be defined as

$$|G\rangle = \prod_{(a,b) \in E} U^{\{a,b\}} |+\rangle^{\otimes V}$$

where $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and operator $U^{\{a,b\}}$ stands for the controlled-Z gate between qubits a and b . With this graph state, MBQC can be carried by a pattern of Z-measurements and equatorial measurements with specific measurement angles. Then, we can obtain the measurement pattern for this graph state, which is generated by combining the graph state with the measurement basis of all qubits.

Figure 1 exploits an existing translation algorithm [2] to translate the sample circuit (Figure 1(a)) to the graph state (Figure 1(b)). The details of the algorithm can be found in [2]. In Figure 1(b), directed edges indicate the dependency between qubits, and qubits with 'in', 'out', or 'in/out' act as the input or output qubits in the given circuit. Also, qubits are marked by the different measurement bases in the graph state.

Fusion and fusion graph in MBQC. The fusion operation is a native measurement operation in linear optics for MBQC. The typical fusion with a success rate of 75% is called Bell fusion, which applies the two-qubit measurement in Bell state projection [19] to project a two-qubit state to an entangled state. The fusion used in this paper is in XZ- and ZX-basis, which can connect one graph state to the other one. Figure 1 demonstrates how one fusion connects

two graph states. The fusion performed on qubits 2 and 3 eliminates these two qubits and attaches qubit 1 to qubit 4, merging two two-qubit graph states into one two-qubit graph state. Recent studies report that the success rate of single fusion can be improved further (e.g., improved from 75% to 90% in some cases [17]).

The fusion graph is to bridge the gap between the graph state and the photonic hardware — each node in the graph state is one qubit state while each node in the photonic hardware is one three-qubit resource state. For the generated fusion graph (Figure 1(c)), each node is a resource state that contains three entangled qubits; and each edge indicates the fusion operation that connects two resource states. During the transformation, when there exist no sufficient photonic qubits to be grouped into multiple resource states, the new qubits need to be introduced so that each photonic qubit belongs to one resource state.

Photonic hardware in MBQC. For Figure 1(d), nodes and edges in the photonic hardware represent resource states and fusions, respectively. All resource states of photonic hardware at one time slot are referred to as one physical layer [7]. A mapped quantum circuit may delay the photonic qubits in allocated resource states to future time slots, resulting in occupying multiple layers. In this paper, the number of physical layers occupied before and after circuit cutting is reported as the physical depth.

2.2 Wire Cutting in Quantum Circuit

Wire cutting is a powerful tool for decomposing an n -qubit quantum circuit into multiple smaller ones (referred to as subcircuits) so that each can fit on k -qubit quantum devices ($k < n$). For example, the sample quantum circuit in Figure 2 can be wire cut into two separated subcircuits c_1 and c_2 .

Since the unitary matrix of any quantum gate can be decomposed into a set of orthonormal matrix bases, the result of the quantum circuit can be obtained from post-processing the results from two subcircuits. For qubit A in c_1 , multiple measurements in different basis are performed on the upstream vertex of this cutting to generate three results. Thus, qubit A is called the measurement qubit in this cutting. On the other hand, in circuit c_2 , four various initialized states are assigned to the initialization qubit A', the downstream vertex of the cutting. It leads to four different output values of c_2 . In the end, all these results are combined to reconstruct the result of the original circuit. More details about wire cutting and reconstruction can be referred to [18].

It's important to note that the post-processing overhead increases exponentially with the number of cuts. Thus, finding a cutting solution with a small number of cuts is critical for mitigating the post-processing overhead.

3 MOTIVATION

Since fusion is a highly error-prone and low-fidelity operation in quantum computing, there exists a great necessity to reduce the number of fusions for MBQC. A careful study of MBQC compilation [7] reveals that the number of fusions in mapped photonic hardware comes from two sources: (1) The larger the quantum circuit is, the more fusions the photonic hardware needs. (2) The more complicated geometry one quantum circuit owns, the more extra

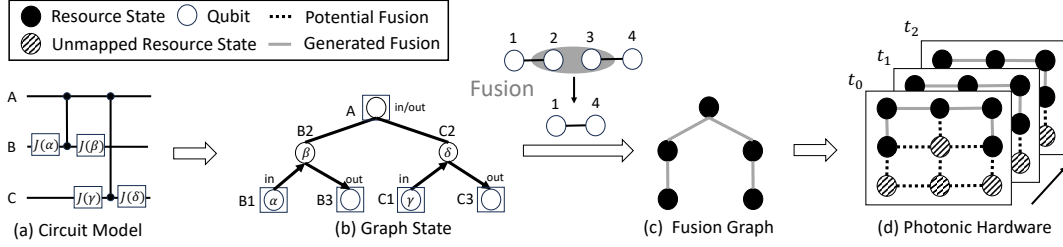


Figure 1: The compilation procedure of MBQC.

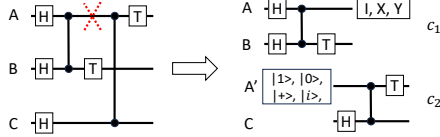


Figure 2: Wire cutting splits one three-qubit circuit into two two-qubit subcircuits.

fusions the mapping algorithm introduces to map its fusion graph to the photonic hardware.

In this paper, we are motivated to exploit wire cutting to decompose a large quantum circuit into several smaller subcircuits such that each subcircuit (1) has fewer circuit gates and thus needs fewer fusions; and (2) has simpler geometry and thus introduces fewer extra fusions during hardware mapping. However, the more cuts are applied, the higher the post-processing overhead is.

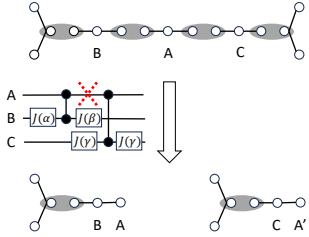


Figure 3: An example on how wire cutting reduces fusions. The original circuit requires four fusions in the fusion graph while two subcircuits generated from one wire cutting only introduce one fusion in each.

Figure 3 illustrates that cutting a large circuit into two smaller circuits reduces fusions in each subcircuit and improves the overall fidelity. For the sample circuit in Figure 3, we need four fusions in the fusion graph. After wire cutting, each of the two subcircuits needs one fusion only. Assuming the success rate of one fusion is 90%, wire cutting improves the fidelity from 65.6% to 90.0%.

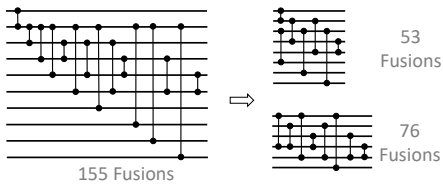


Figure 4: Applying four wire cuts on a complex circuits with 155 fusions after mapping generates two subcircuits, which only have 53 fusions and 76 fusions, respectively.

Figure 4 illustrates that cutting can reduce the extra fusion operations when mapping the fusion graph to photonic hardware. While

a node on both fusion graph and photonic hardware represents a resource state, the photonic hardware exhibits layout constraints – one node has at most four neighbors on a 2D physical layer while each corner node has two neighbors.

Consequently, mapping a high-degree node in a graph state introduces extra fusions for the *routing* purpose, i.e., to connect different computation components. In the figure, eight CZ gates share the same control qubit q1 simultaneously, leading to a tree-shape graph (e.g. Figure 1(b)) and extra fusions for the routing. After performing four wire cuts, we obtain two subcircuits where the maximum number of CZ gates sharing the same control qubit is three. As a result, the numbers of fusions in the two subcircuits are 53 and 76, respectively. The sum of them is 17% smaller than the original number, indicating a significant reduction of extra fusions.

Existing wire cutting schemes, e.g., CutQC [18] and Clifford-based cutting [8], focus on decomposing a quantum circuit of width n to run on quantum computers with k physical qubits ($n > k$). They are fusion-oblivious when searching for cutting solutions. However, for wire cutting on photonic MBQC, our cutting goal is i) to balance fusions among different subcircuits and ii) to reduce the extra fusions during mapping. Thus, a novel wire cutting approach is needed to meet our design goal.

4 DESIGN

In this paper, we develop FCM, a Fusion-aware Cutting approach for MBQC, that cuts a large quantum circuit into smaller subcircuits with simpler geometry. We formulate the problem using MIP model so as to find the best cutting decisions under different problem settings. Once FCM splits the original circuit into subcircuits, it deploys each subcircuit onto photonic hardware using OneQ, and then combines the results through post-processing.

Intuitively, FCM strives to minimize the number of fusions in each subcircuit and the number of cuts at the same time. The former is to improve the overall computation fidelity while the latter is to reduce the post-processing overhead. Our observations reveal that the number of subcircuits largely determines the number of fusions in each subcircuit. The elimination of extra fusions after cutting, while being important, plays a secondary role. In Figure 4, extra fusions are reduced by nearly 20%. However, cutting an n -fusion circuit to four subcircuits instead of two subcircuits reduce the number of fusions from the range of $0.5n$ to the range of $0.25n$. Unfortunately, the more number of subcircuits FCM cuts the original circuit to, the more number of cuts it requires. Therefore, we may need to adjust our optimization goals under different settings.

We next present an illustrating example, assuming we are to cut a 2000-fusion circuit into two subcircuits under three different settings, we have following three kinds of solutions:

- (i) **Balanced-FCM (B-FCM)**: If our goal is to improve computation fidelity with a reasonable number of cuts, we seek for the best trade-off between evenly distributing fusions into subcircuits and reducing the number of cuts.
- (ii) **Threshold-FCM (T-FCM)**: If the success rate of a subcircuit that has around 1500 fusions is acceptable, we strive to minimize the post-processing cost while we relax the balance constraint and focus on minimizing the number of cuts.
- (iii) **Fusion-FCM (F-FCM)**: If we strive to improve fidelity as much as possible regardless of post-processing overhead, we may cut it to two subcircuits only focusing on balancing the fusions.

4.1 FCM

We model the quantum circuit as a directed acyclic graph G where the vertices set $V = \{v_1, \dots, v_{n_V}\}$ represents multi-qubit gates modeled and the edges set $E = \{e_1, \dots, e_{n_E}\}$ represents circuit wires. We focus on two-qubit gates when finding the cutting solution because single-qubit gates do not affect the circuit connectivity and do not introduce extra fusions during hardware mapping. Note that FCM also works for multi-qubit gates that involve more than two qubits as all the multi-qubit gates can be decomposed to the native gate set before the execution. We next list the parameters, the constraints, and the objective function in the MIP model.

4.1.1 Constant Parameters. In FCM, the parameter n_C specifies the number of subcircuits generated after cutting, i.e., the set of subcircuits are $C = \{c_1, \dots, c_{n_C}\}$. Also, to distinguish gates that cause extra fusions in the mapping phase from those that do not, we assign a fusion weight f_v to each two-qubit gate — its value depends on whether the gate is sharing the same control qubit with others. For those do not, we have $f_v = 1$. For gates sharing the same control qubit, firstly we count the number of them and those not. Then, we map the given circuit to physical layers and generate fusions. Next, we search for f_v for gates sharing the same control qubits to enable the sum of fusion weights for all two-qubit gates equal to the number of generated fusions.

4.1.2 Variables. First, we model gates and wires in each subcircuit with the following variables.

$$y_{v,c} = \begin{cases} 1 & \text{if vertex } v \text{ is in subcircuit } c \\ 0 & \text{otherwise} \end{cases}, \forall v \in V, \forall c \in C$$

$$x_{e,c} = \begin{cases} 1 & \text{if edge } e \text{ is cut by subcircuit } c \\ 0 & \text{otherwise} \end{cases}, \forall e \in E, \forall c \in C$$

We then define the total fusion weight for each subcircuit and all subcircuits by the fusion weight of each gate as below:

$$W_c = \sum_{v \in V} f_v * y_{v,c}, \quad \forall c \in C \quad (1)$$

$$W = \sum_{c \in C} W_c \quad (2)$$

4.1.3 Constraints. We next present the constraints that the variables must satisfy when searching for the solution. First, each vertex must belong to one and the only one subcircuit, which can be defined as below:

$$\sum_{c \in C} y_{v,c} = 1, \quad \forall v \in V \quad (3)$$

Then, for each edge, it can only exist in one of the subcircuits. For an edge that is not cut, its two end vertices a and b belong to the same subcircuit. However, if an edge is cut by a subcircuit, a and b would belong to two different subcircuits. As a result, we have the following linear constraints:

$$\begin{aligned} x_{e,c} &\leq y_{e_a,c} + y_{e_b,c} \\ x_{e,c} &\geq y_{e_a,c} - y_{e_b,c} \\ x_{e,c} &\geq y_{e_b,c} - y_{e_a,c} \\ x_{e,c} &\leq 2 - y_{e_a,c} - y_{e_b,c} \end{aligned} \quad (4)$$

For the fusion weight of each sub-circuit, we define an upper-bound threshold T , which can be customized for different problem settings. Thus, we have the following constraint definition:

$$W_c \leq T, \quad \forall c \in C \quad (5)$$

4.1.4 Objective Function. Since our main goal is to reduce the number of fusions in each subcircuit and to balance the fusions among subcircuits, we choose to evenly distribute gates sharing the same control qubits with other gates into different subcircuits. These gates tend to introduce more extra fusions as we discussed in Section 3. Because W is proportional to the one in the original circuit, according to AM-GM Inequality [9], we achieve minimized deviation among W_c values if their product is maximized. Hence, we choose the following objective function to maximize in order to find a better cutting solution:

$$L \equiv \prod_{c \in C} W_c \equiv \prod_{c \in C} \sum_{v \in V} f_v * y_{v,c} \quad (6)$$

It is known that the multiplication computation of variables is non-linear, which is hard to solve for the MIP model. To address that, we apply the piece-wise linear approximation of log function in Gurobi [20] to simplify this objective function as follows:

$$L \equiv \sum_{c \in C} \log \sum_{v \in V} f_v * y_{v,c} \quad (7)$$

It's notable that this piece-wise linear approximation of log function can be solved efficiently, similar as the linear function. By using the log function, we can force the first term in equation 9 to be in the same order of magnitude as the number of cuts in most cases. Besides, because post-processing overhead grows as more cuts are applied, we should also consider the number of cuts in the objective function so that we can reduce the fusion of all subcircuits with fewer cuts. As a result, the number of cuts and the final objective function is defined below:

$$K = \frac{1}{2} \sum_{c \in C} \sum_{e \in E} x_{e,c} \quad (8)$$

$$L \equiv \alpha \sum_{c \in C} \log \sum_{v \in V} f_v * y_{v,c} + \beta K \quad (9)$$

where α and β are meta parameters. Overall, the MIP model can be formulated as:

$$\begin{aligned} & \text{maximize objective } L(\text{Eq. 9}) \\ & \text{s.t. constraint Eqs. (3, 4, 5)} \end{aligned} \quad (10)$$

4.2 Case Studies

By choosing different meta parameters, we can adapt our FCM model to meet different optimization goals.

- (i) B-FCM: We choose $\alpha=1, \beta=-1$, indicating that we take both computation fidelity and post-processing cost into consideration. We set $T=W$, indicating we do not put a hard threshold on the number of fusions per subcircuit.
- (ii) T-FCM: We set $\alpha=0$ and $\beta=-1$, indicating we no longer emphasize balancing the fusions across subcircuits. Instead, we set T to a value determined by the single fusion success rate. In the experiments, we choose $T=500$ and 1000 , to study its effectiveness on choosing different cutting decisions.
- (iii) F-FCM: We set $\alpha=1$ and $\beta=0$, indicating the post-processing overhead is ignored. While we set $T=W$, it has little impact as our goal is to balance fusions across subcircuits, which achieves the best computation fidelity with a given number of subcircuits.

5 EVALUATION

5.1 Benchmarks and Metrics

We conduct the evaluation of our methods using five benchmarks, including supremacy [15], Approximate Quantum Fourier Transform (aqft) [1], Bernstein–Vazirani (bv) [3], Quantum Approximate Optimization Algorithm (qaoa) [4], and random. The depth of supremacy and random is configured as 20 and 40, respectively. For bv, we randomly generate secret strings with the number of qubits as the length of the string. The quantum circuits in qaoa and random are generated randomly without any restriction, while gates in other benchmarks are generated by repeated specific patterns. In the evaluation, we set the area size of each physical layer as 16×16 and evaluate each benchmark with three different numbers of qubits as 16, 25, and 36. We apply the mathematical optimization solver Gurobi [20] as MIP model solver backend and set the maximum timeout of solving the MIP model as 300 seconds.

We compare our methods against the baseline, which is to take the original circuit as input and deliver the mapping result using OneQ. We use three metrics to evaluate our method: the number of fusions, the physical depth, and the number of cuts. For the number of fusions and the physical depth, we only report the maximum value of them among all subcircuits as the maximum determines the fidelity as we discussed in Section 3.

5.2 Result

5.2.1 B-FCM. Figure 5 reports the reduction of fusions achieved by B-FCM when cutting the original circuit into k subcircuits ($k=2, 3$, and 4). From the figure, the average reductions of the maximum number of fusions are 50.6%, 66.2%, and 75.6% when $k=2, 3$, and 4 , respectively. Theoretically, cutting a circuit into two subcircuits, i.e., $k=2$, can achieve reduction higher than 50% if the fusions are perfectly balanced. Similar theoretical lower bound of reductions 66.7% and 75% exist for $k=3$ and 4 . While our averages are close to their theoretical average lower bounds, B-FCM on quantum circuits

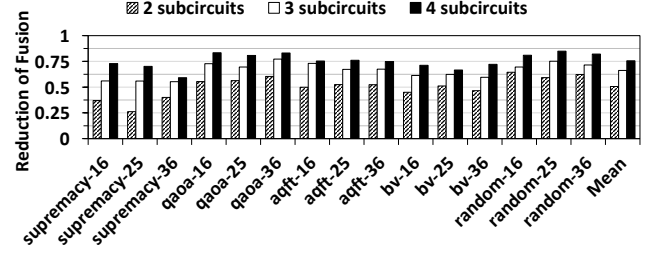


Figure 5: Reduction of the maximum number of fusions achieved by B-FCM on five benchmarks (ABC-# indicates benchmark ABC with # numbers of qubits).

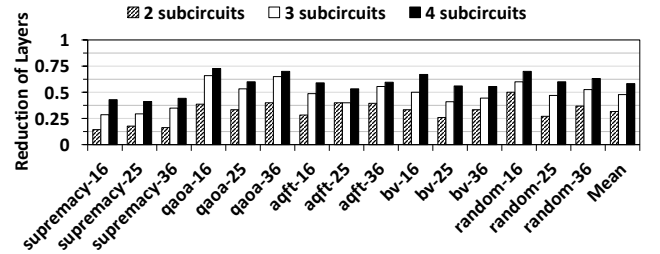


Figure 6: Reduction of the maximum number of physical depths achieved by B-FCM on five benchmarks (ABC-# indicates benchmark ABC with # numbers of qubits).

having random patterns, e.g., qaoa and random, achieves higher reductions. This is because these circuits have more cases in which multiple gates share the same control qubit and thus require more extra fusions during hardware mapping. B-FCM distributes these gates to different subcircuits, leading to more reductions.

Other benchmarks, e.g., supremacy-25, achieve 25% reduction for $k=2$, which is lower than the theoretical 50% for a balanced fusion split. B-FCM chooses the reported cutting solutions because it considers both the balanced fusion split and the number of cuts. We evaluate the best effectiveness of FCM on fusion reduction by F-FCM, which is demonstrated in section 5.2.2.

Figure 6 reports the reductions of the maximum physical depth among all subcircuits with different numbers of subcircuits k . From the figure, the average reductions of the physical depth are 31.6%, 47.8%, and 58.3% when $k=2, 3$, and 4 , respectively. For qaoa and random, B-FCM can achieve 55.4% and 51.8% reductions, respectively. They are bigger than the overall average 45.9%. This is because when mapping the fusion graph to physical layers, to accommodate the cases that some gates share the same control qubits, qubits tend to be delayed to upcoming layers to avoid extra fusions. Therefore, cutting simplifies the subcircuit and thus reduces more physical layers.

Table 1 presents the number of cuts applied by B-FCM on five benchmarks to obtain subcircuits of different numbers. In most cases except bv, the number of cuts increases as the number of qubits or the number of subcircuits grows. This is because in bv, all the two-qubits gates are using different qubits as the same control qubit and the same qubit as the target qubit. Consequently, cuts required to decompose the original circuit into multiple subcircuits are all performed on the target qubits, and the number of cuts always equals to the number of subcircuit minus one.

Regarding the time cost, the average searching time on all benchmarks is 9.06 seconds, 104.63 seconds, and 195.01 seconds where the number of subcircuits is 2, 3, and 4, respectively. For all cases, B-FCM manages to provide solutions with a significant reduction of fusions within the given period.

Benchmark	#Qubit	#Cuts used for different #Subcircuits			
		2	3	4	
supremacy	16	10	15	20	
	25	12	19	25	
	36	15	23	29	
qaoa	16	14	23	32	
	25	18	26	36	
	36	19	33	44	
aqft	16	5	10	15	
	25	5	10	15	
	36	6	12	18	
bv	16	1	2	3	
	25	1	2	3	
	36	1	2	3	
random	16	14	21	31	
	25	17	30	41	
	36	22	27	56	

Table 1: The number of cuts applied by B-FCM to decompose circuits of various widths into subcircuits.

5.2.2 F-FCM. When the optimization goal is the fusion reduction, F-FCM reduces fusions by 59.6%, 76.0% and 83.6% on average (up to 69.1%, 85.5%, and 90.7%) for $k=2, 3$, and 4, respectively. The smallest improvement is 55.8% reduction for bv-25 with $k=2$, which is better than the theoretical 50% due to the elimination of extra fusions. Regarding the reduction of physical depths, F-FCM can achieve 42.5%, 56.5%, and 67.8% on average for $k=2, 3$, and 4, respectively.

Note, choosing $\beta=0$ results in significantly more numbers of cuts, the average numbers of cuts increase by 1.68x, 1.49x, and 1.38x when $k=2, 3$, and 4, respectively.

5.2.3 T-FCM. In this section, we evaluate T-FCM only on benchmarks random and qaoa since they are the most complicated ones generating large numbers of fusions. Table 2 reports the reduction of cuts and the number of subcircuits under different thresholds T . A threshold T indicates that the fidelity is acceptable if the number of fusions for a mapped photonic hardware is below T .

When setting the threshold to be 1000 and 500, T-FCM reduces cuts by 12.9% and 15% on average over B-FCM, respectively. For benchmark random-16, since the number of fusions in the original circuit is below 1000, there is no need to perform any cut if $T=1000$.

5.2.4 Comparison to CutQC. CutQC [18] was proposed to cut a circuit into smaller subcircuits so that they can run on superconducting quantum computers with fewer qubits. The number of subcircuits and the number of quantum devices with limited size are constants. Naively adapting CutQC to solve the problem for MBQC leads to sub-optimal results.

A comparison between B-FCM and CutQC shows that B-FCM outperforms CutQC by 40.1%, 24.9%, and 15.6% on average when $k=2, 3$, and 4, respectively. The two schemes have similar numbers of cuts while B-FCM has one or two more cuts for some large circuits, due to choosing different cutting decisions.

5.2.5 Result Correctness. Based on the wire-cutting theory and the re-construction process introduced and proved in [18], our cutting solution outputs the same result as the original circuit.

Benchmark	#Qubit	Reduction of #Cuts (#Subcircuits) under different thresholds T	
		1000	500
random	16	- (-)	7.1% (2)
	25	15.0% (2)	12.2% (3)
	36	14.3% (2)	11.6% (4)
qaoa	16	13.3% (2)	21.7% (3)
	25	11.1% (2)	19.2% (3)
	36	10.5% (2)	18.2% (4)

Table 2: With different threshold T , the reduction of #Cuts gained by T-FCM compared to B-FCM on qaoa and random.

6 CONCLUSION

This paper is the first to present the insight that wire cutting can be leveraged to reduce fusions in MBQC. We propose FCM to decompose a large circuit into subcircuits for improved computation fidelity. It leverages the fusion weight to distinguish the two-qubit gates that tend to generate more extra fusions and constructs an MIP model to find the optimal cut solution under three typical settings. With appropriate adjustments, FCM can be adapted to meet the cutting demands with special design goals.

ACKNOWLEDGMENTS

The authors would like to thank the anonymous reviewers for their constructive feedback and suggestions. This work is supported in part by PQI Community Collaboration Award #007913, NSF grants #2011146, #2154973, #1725657, #1910413, and #2312157.

REFERENCES

- [1] Adriano Barenco *et al.* 1996. Approximate quantum Fourier transform and decoherence. *Phys. Rev. A* 54 (1996), 139–146. Issue 1.
- [2] Aleks Kissinger *et al.* 2020. PyZX: Large Scale Automated Diagrammatic Reasoning. *Electronic Proceedings in Theoretical Computer Science* 318 (2020), 229–241.
- [3] Ethan Bernstein *et al.* 1993. Quantum complexity theory. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*. 11–20.
- [4] Edward Farhi *et al.* 2014. A Quantum Approximate Optimization Algorithm.
- [5] Fabian Ewert *et al.* 2014. 3/4-Efficient Bell Measurement with Passive Linear Optics and Unentangled Ancillae. *Phys. Rev. Lett.* 113 (2014), 140403. Issue 14.
- [6] Hector Bombin *et al.* 2021. Interleaving: Modular architectures for fault-tolerant photonic quantum computing. *arXiv* (2021).
- [7] Hezi Zhang *et al.* 2023. OneQ: A Compilation Framework for Photonic One-Way Quantum Computation. In *ISCA, 2023*. 12:1–12:14.
- [8] Kaitlin N. Smith *et al.* 2023. Clifford-based Circuit Cutting for Quantum Simulation. In *ISCA, 2023*. 1–13.
- [9] Limin Zou *et al.* 2015. Improved arithmetic-geometric mean inequality and its application. *Journal of Mathematical Inequalities* (2015), 107–111.
- [10] Michael A. Perlin *et al.* 2021. Quantum circuit cutting with maximum-likelihood tomography. *npj Quantum Information* 7, 1 (2021), 1–8.
- [11] Mercedes Gimeno-Segovia *et al.* 2015. From Three-Photon Greenberger-Horne-Zeilinger States to Ballistic Universal Quantum Computation. *Phys. Rev. Lett.* 115 (2015), 020502. Issue 2.
- [12] M. H. Devoret *et al.* 2013. Superconducting Circuits for Quantum Information: An Outlook. *Science* 339, 6124 (2013), 1169–1174.
- [13] Pieter Kok *et al.* 2007. Linear optical quantum computing with photonic qubits. *Reviews of Modern Physics* 79, 1 (2007), 135–174.
- [14] Robert Raussendorf *et al.* 2003. Measurement-based quantum computation on cluster states. *Phys. Rev. A* 68 (2003), 022312. Issue 2.
- [15] Sergio Boixo *et al.* 2018. Characterizing quantum supremacy in near-term devices. *Nature Physics* 14, 6 (2018), 595–600.
- [16] Sara Bartolucci *et al.* 2023. Fusion-based quantum computation. *Nature Communications* 14, 1 (2023), 912.
- [17] Thomas Kilmer *et al.* 2019. Boosting linear-optical Bell measurement success probability with predetection squeezing and imperfect photon-number-resolving detectors. *Phys. Rev. A* 99 (2019), 032302. Issue 3.
- [18] Wei Tang *et al.* 2021. CutQC: using small Quantum computers for large Quantum circuit evaluations. In *ASPLOS, 2021*. 473–486.
- [19] W. P. Grice. 2011. Arbitrarily complete Bell-state measurement using only linear optical elements. *Phys. Rev. A* 84 (2011), 042331. Issue 4.
- [20] Gurobi Optimization LLC. 2023. Gurobi Optimizer Reference Manual. <http://www.gurobi.com>.